

Mobile Networks and IoT

2026/2027

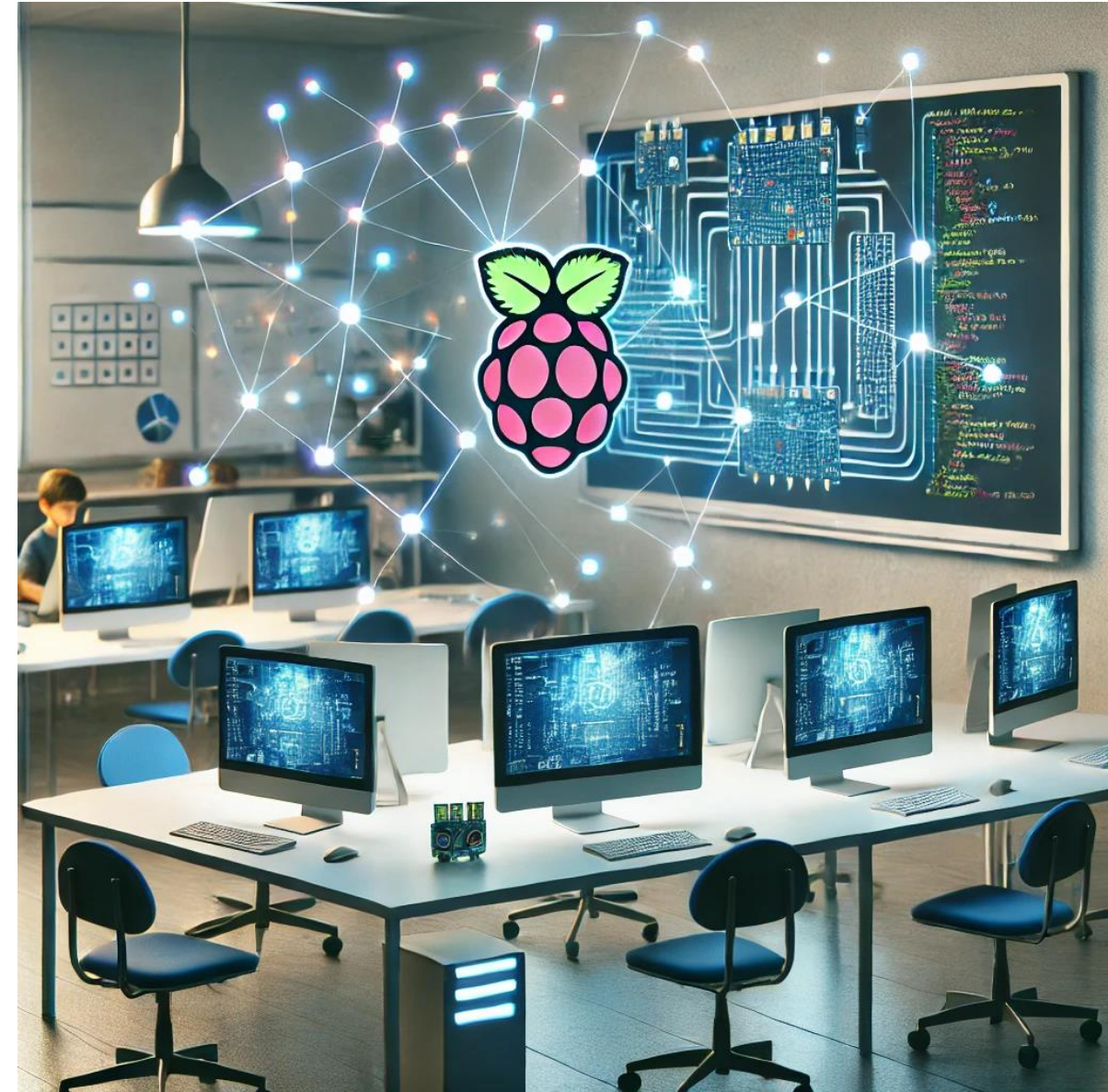
PL Class 02 – Introduction to MQTT

Vasco Pereira (vasco@dei.uc.pt)

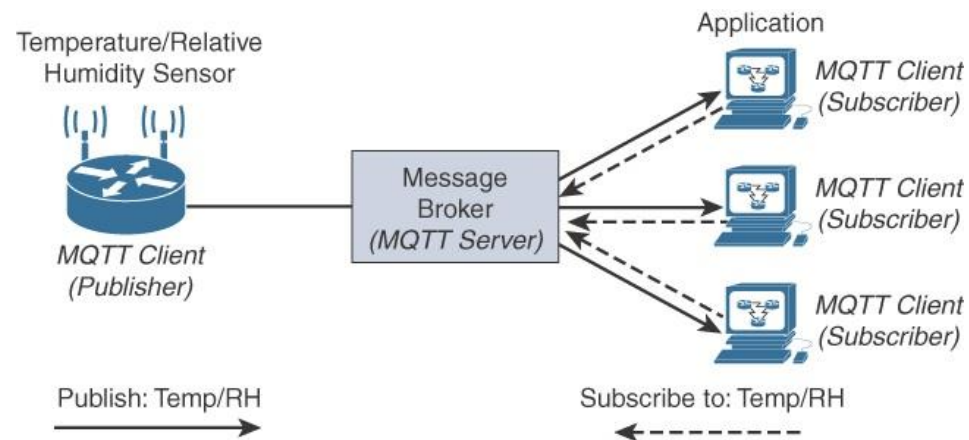
Dep. Eng. Informática da Faculdade de Ciências e Tecnologia da Universidade de Coimbra

Introduction to MQTT

Connecting our classroom



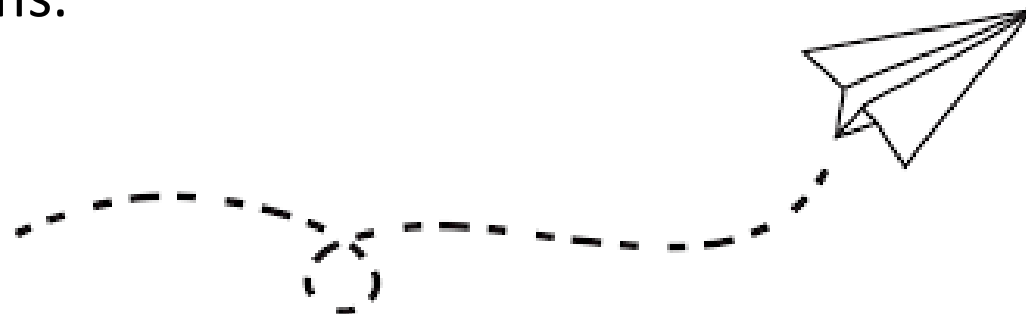
- Message Queuing Telemetry Transport (MQTT)
 - At the end of the 1990s, engineers from IBM and Arcom were looking for a reliable, lightweight, and cost-effective protocol to monitor and control a large number of sensors and their data from a central server location. Their research resulted in the development and implementation of the MQTT.
 - An extremely simple protocol with only a few options was designed, with considerations for constrained nodes, unreliable WAN backhaul communications, and bandwidth constraints with variable latencies. These were some of the rationales for the selection of a client/server and publish/subscribe framework based on the TCP/IP architecture.



What is MQTT?

4

- Message Queuing Telemetry Transport (MQTT) - main characteristics:
 - A lightweight communication protocol.
 - Designed for constrained devices.
 - Efficient for low-bandwidth, unreliable networks.
 - Ideal for sending small messages efficiently.
 - Connects diverse devices.
 - Uses a publish/subscribe model.
 - Enables one-to-many/many-to-one/many_to_many communications.
 - MQTT provides push-based communication. This push is made possible via TCP connections.



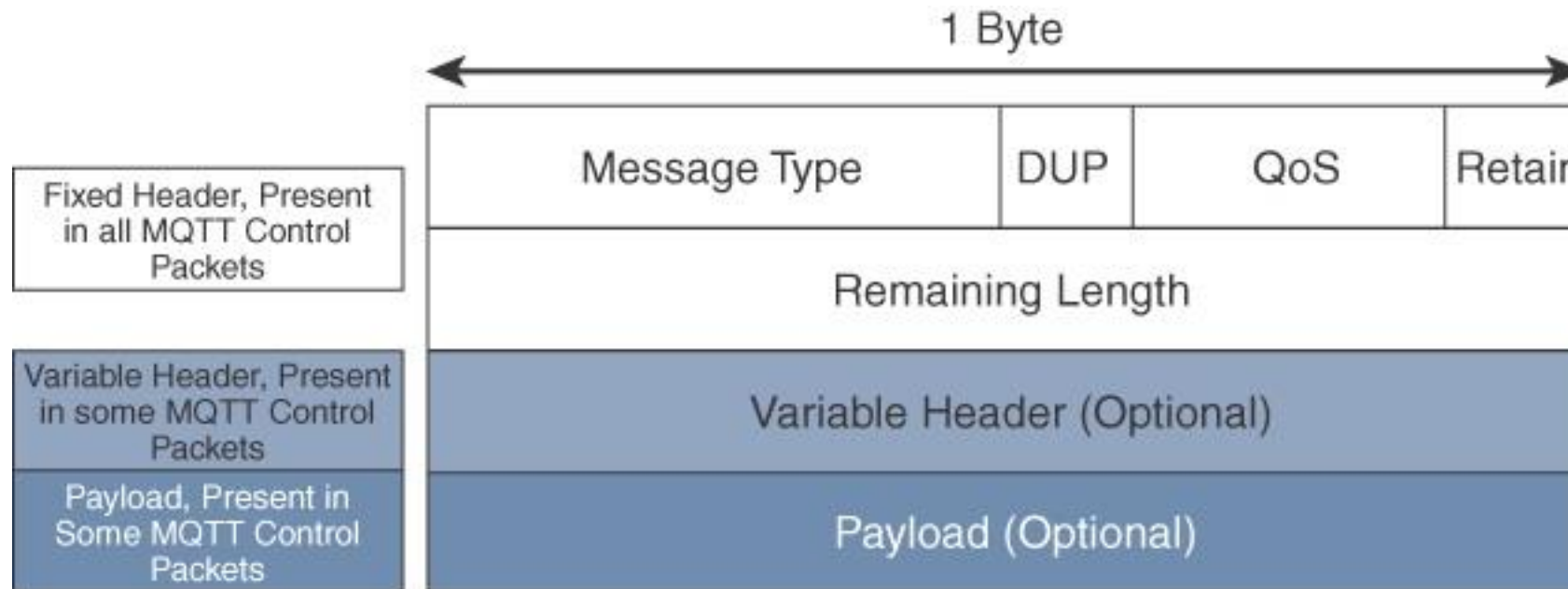
- MQTT uses a broker-based architecture.
 - The sensor can be set to be an MQTT publisher (publishes information).
 - The application that needs to receive the information is set as the subscriber.
 - Any intermediary system can be set as a broker to relay the information between the publisher and the subscriber(s).
- MQTT clients can subscribe to all data or specific data from the information tree of a publisher.
- MQTT runs over TCP.
 - A consequence of the reliance on TCP is that an MQTT client typically holds a connection open to the broker at all times. This may be a limiting factor in environments where loss is high or where computing resources are limited.

- Is an OASIS (Organization for the Advancement of Structured Information Standards) standard (www.oasis-open.org/)
- It is also an ISO standard (ISO/IEC PRF 20922)
- Public and royalty-free license
- Libraries available for Android, Arduino, Pi, C, C++, C#, Java...
- Latest version: MQTT V5.0 (<http://mqtt.org/>)
- IANA reserved ports:
 - TCP port 1883 (MQTT)
 - TCP port 8883 (MQTT over SSL)

MQTT

Packet

- MQTT is a lightweight protocol because each control packet consists of a 2-byte fixed header with optional variable header fields and optional payload. A control packet can contain a payload up to 256 MB.



- The **Message Type** defines the type of control packet.

Message Type	Value	Flow	Description
CONNECT	1	Client to server	Request to connect
CONNACK	2	Server to client	Connect acknowledgement
PUBLISH	3	Client to server Server to client	Publish message
PUBACK	4	Client to server Server to client	Publish acknowledgement
PUBREC	5	Client to server Server to client	Publish received
PUBREL	6	Client to server Server to client	Publish release
PUBCOMP	7	Client to server Server to client	Publish complete
SUBSCRIBE	8	Client to server	Subscribe request
SUBACK	9	Server to client	Subscribe acknowledgement
UNSUBSCRIBE	10	Client to server	Unsubscribe request
UNSUBACK	11	Server to client	Unsubscribe acknowledgement
PINGREQ	12	Client to server	Ping request
PINGRESP	13	Server to client	Ping response
DISCONNECT	14	Client to server	Client disconnecting

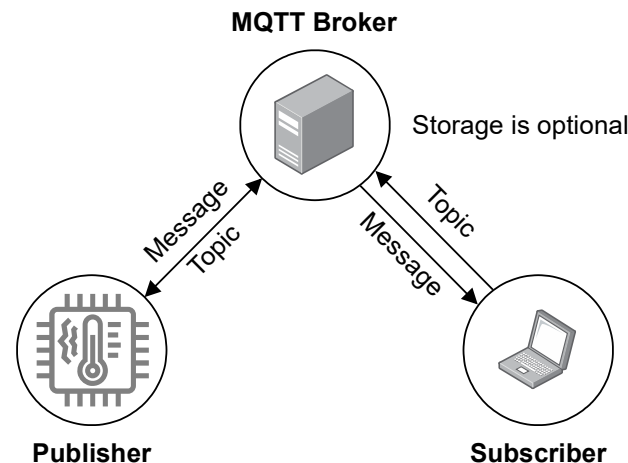
- The **DUP** (Duplication Flag), when set, allows the client to notate that the packet has been sent previously, but an acknowledgement was not received.
- The QoS header field allows for the selection of three different QoS levels.
 - **QoS 0**: best-effort and unacknowledged data service, referred to as “at most once” delivery. The publisher sends its message one time to a server, which transmits it once to the subscribers. No response is sent by the receiver, and no retry is performed by the sender. The message arrives at the receiver either once or not at all.
 - **QoS 1**: ensures that the message delivery between the publisher and server and then between the server and subscribers occurs at least once. This level guarantees “at least once” delivery.
 - **QoS 2**: highest QoS level, used when neither loss nor duplication of messages is acceptable. This level provides a “guaranteed service” known as “exactly once” delivery, with no consideration for the number of retries as long as the message is delivered once.

- The **Retain flag** (only found in PUBLISH messages) notifies the server to hold onto the message data.
 - The broker stores only one retained message per topic.
 - By enabling the retention of the most recent message on a topic, subscribers can stay informed about the current state, even during periods of inactivity.
 - This allows new subscribers to instantly receive the last known value without having to wait for the next update from the publisher.
- The **Remaining Length** field specifies the number of bytes in the MQTT packet following this field.
 - Remaining length is between 1 and 4 bytes
 - The maximum length size is 256MB.
 - Small packets less than 127 bytes have a 1 byte packet length field.
 - Packets larger than 127 and less than 16383 will use 2 bytes, etc.
 - Note: 7 bits are used with the 8th bit being a continuation bit.
 - 7bits x4 =28 bits => 2^{28} =256MB

- MQTT sessions between each client and server consist of four phases:
 - Session establishment
 - Authentication
 - Data exchange
 - Session termination
- Each client connecting to a server has a unique client ID, which allows the identification of the MQTT session between both parties. When the server is delivering an application message to more than one client, each client is treated independently.

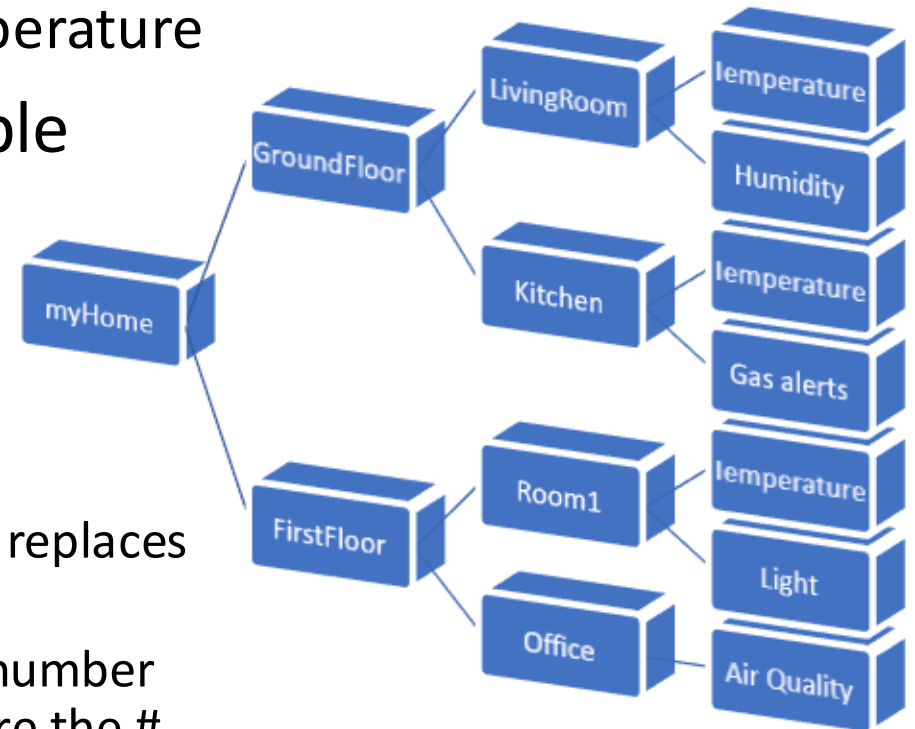
- Subscriptions to resources generate SUBSCRIBE/SUBACK control packets, while unsubscription is performed through the exchange of UNSUBSCRIBE/UNSUBACK control packets.
- Graceful termination of a connection is done through a DISCONNECT control packet, which also offers the capability for a client to reconnect by re-sending its client ID to resume the operations.

- Publish/subscribe model
 - **Publishers:** send (write) messages to topics.
 - **Subscribers:** receive (read) messages from specific topics subscribed.
 - **Broker:** handles the communication between publishers and subscribers. Receives, filters, and distributes messages.
 - The broker acts as a simple switchboard, accepting messages from publishers of specified topics and sending them to subscribers of those topics.
 - A MQTT client may act as subscriber, as publisher, or as both



- The broker decouples publishers and subscribers
 - Space decoupling
 - Publisher and subscriber are not aware of each other's network location and do not exchange information such as IP addresses or port numbers.
 - Time decoupling
 - The publisher and subscriber do not need to run or have network connectivity at the same time.
 - Synchronization decoupling
 - Publishers and subscribers can send or receive messages without interrupting each other. E.g.: the subscriber does not have to wait for the publisher to send a message.

- Topics are like "channels" for messages.
- They use a hierarchical structure.
 - e.g., myHome/GroundFloor/LivingRoom/Temperature
- Wildcards (#, +) allow subscribing to multiple topics.
 - # (multi-level): Matches all sub-levels.
 - + (single-level): Matches one level.
 - E.g.,
 - *myHome/GroundFloor+/Temperature* => the '+' replaces any one level
 - *myHome/Groundfloor/#* => the '#' replaces any number of levels that begin with the pattern specified before the #
- Used for filtering and routing messages.



- Central server that manages message flow.
- Responsible for routing messages to subscribers.
- Can handle many concurrent connections.
- Popular brokers: Mosquitto, HiveMQ, EMQX.

- MQTT clients that subscribe to a new topic have no insight into when to expect the first message they will receive. An MQTT broker can store a **retained message** that can be sent immediately upon a new MQTT subscription.
 - The broker stores only one retained message per topic (the last retained message published).
 - Each client that subscribes to a topic pattern that matches the topic of the retained message receives the retained message immediately after they subscribe.
 - By enabling the retention of the most recent message on a topic, subscribers can stay informed about the current state, even during periods of inactivity.
 - There is only one way to delete the retained message of a topic. To achieve this, simply publish a retained message with a zero-byte payload to the topic where the retained message is stored.

- An MQTT client can specify to an MQTT broker a message, called the **Last Will**, that will be sent if the MQTT client ungracefully disconnects. This allows for a more graceful system wide notification that a client has been disconnected.
 - When a client connects to a broker, it can specify a Last Will message. This message follows the structure of a regular MQTT message structure
 - The broker stores this message until an ungraceful disconnect from the client.
 - The Last Will allows clients to notify others about their unexpected disconnections.
 - The broker broadcasts the Last Will message to all subscribed clients of the corresponding topic.
 - The broker discards the stored Last Will message if the client disconnects gracefully using the DISCONNECT message.

- Multiple brokers (Message servers) may be connected together, using the bridging functionality.
- This is useful where it is desirable to share information between locations, but where not all the information needs to be shared.
- By defining topic patterns and direction parameters you can control the data flow between the bridged servers.
- For example:
 - Bridge messages with Topic X from Server A to B
 - Bridge messages with Topic Y from both Servers
 - Do not bridge messages with other Topics

- When a local broker is configured to act as a bridge, it becomes an MQTT client to the remote broker and subscribes and publishes to topics on the other broker just like a normal MQTT client.

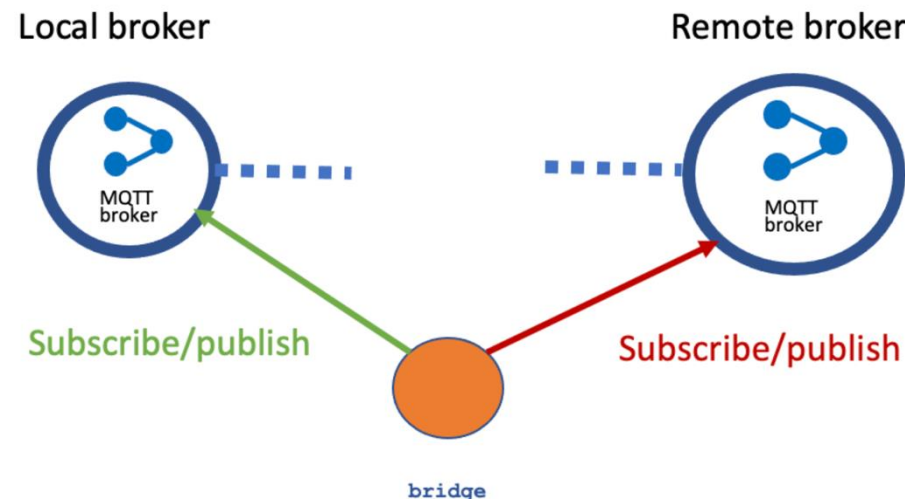


Image from: <https://hackmd.io/@phdunimed/mqttbridging>

- On connection, a client sets the **clean session** flag, which is sometimes also known as the **clean start** flag.
 - If **clean session flag** is set to false, the connection is treated as durable.
- MQTT durable connections (aka durable sessions), allow a client to disconnect and reconnect without losing subscribed topics or queued messages.
 - The broker stores the client's session state, including subscriptions and undelivered messages, and delivers them upon reconnection, ensuring data is not lost and maintaining continuity in communication.
- If **clean session** flag is true, then all subscriptions will be removed for the client when it disconnects.

- IoT devices often handle sensitive data.
- Without security, MQTT networks are vulnerable.
- Protecting against eavesdropping, tampering, and unauthorized access is critical.
- Some security aspects to consider:
 - Authentication: verifying identity;
 - Access Control Lists: granular control over access.
 - TLS Encryption: securing communication by using encryption;

- Securing MQTT connections through Transport Layer Security (TLS) is considered optional because it calls for more resources on constrained nodes.
 - When TLS is not used, the client sends a clear-text username and password during the connection initiation.
 - MQTT server implementations may also accept anonymous client connections (with the username/password being “blank”).
 - When TLS is implemented, a client must validate the server certificate for proper authentication. Client authentication can also be performed through certificate exchanges with the server, depending on the server configuration.

MQTT versions

25

Feature	MQTT 3.1.1	MQTT 5.0	MQTT-SN (Sensor Network)
Standardized	OASIS, ISO/IEC	OASIS, ISO/IEC	Not ISO, specified by IBM & others
Primary Use	General IoT Messaging	Advanced IoT Messaging	Low-power / constrained networks (e.g. Zigbee)
Transport	TCP/IP	TCP/IP	UDP / other non-TCP networks
Broker Required	✓ Yes	✓ Yes	✓ Yes
Target Devices	IoT devices with reliable TCP support	Enterprise-grade IoT, edge/cloud systems	Battery-powered or wireless mesh nodes
Adoption Level	Widely adopted	Gaining adoption	Niche / special use
QoS Levels	0, 1, 2	0, 1, 2	0, 1, 2
Topic Format	Text strings	Text strings	Short topic IDs/aliases (for efficiency)
Message Properties	Limited	Rich user and system-defined props	Minimal (optimized for low power)
Session Control	Clean session flag	Session Expiry, Will Delay, etc.	Lightweight session model
Error Handling	Basic disconnect codes	Extensive reason codes	Minimal error feedback

And now, some exercises...
Go to Assignment!

Thank you! Questions?

